Witryny i Aplikacje Internetowe

Autor: Mateusz Dziedzic

Spis treści

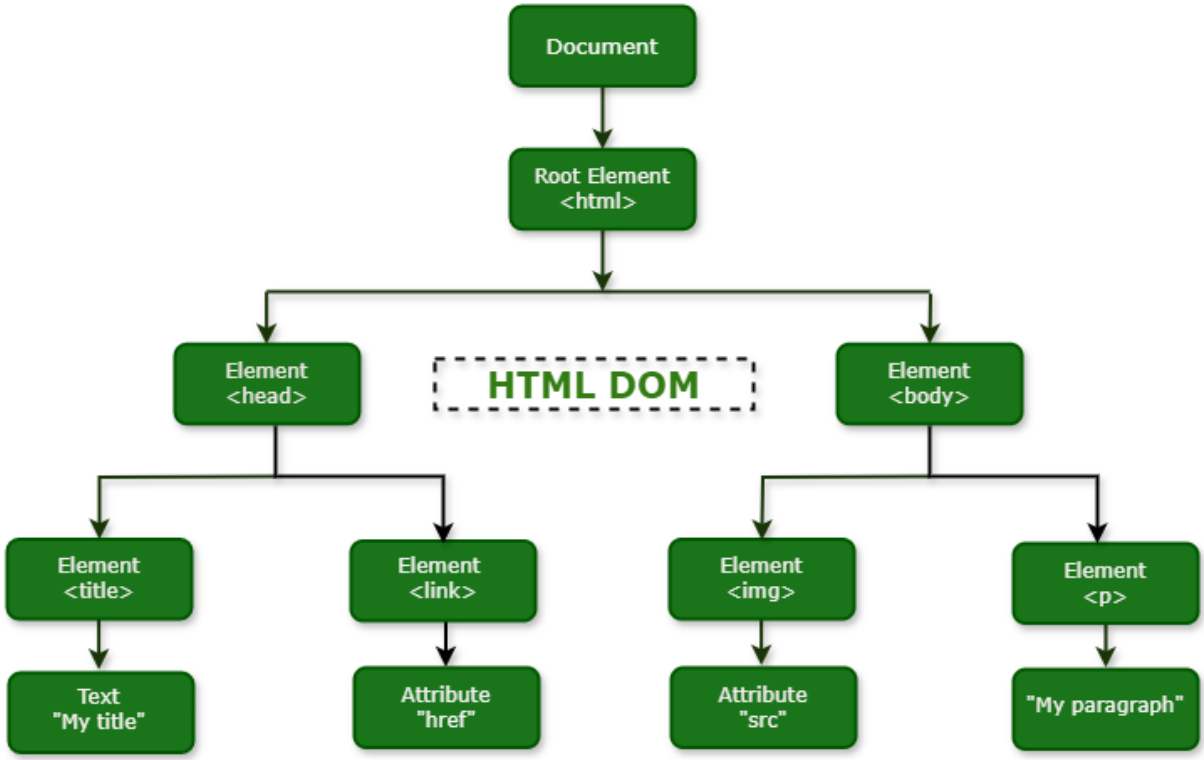
1. Co to jest DOM? 2
Diagram drzewa DOM 2
Dlaczego DOM jest ważny 2
Przykład działania DOM 2
2. Wybieranie elementów HTML 3
Najważniejsze metody: 3
`document.getElementById('id')` 3
`document.getElementsByClassName('class')` 3
`document.getElementsByTagName('tag'` 3
`document.querySelector('selector')` 3
`document.querySelector('selector')` 4
3. Modyfikowanie treści i stylów 4
Zmiana tekstu 4
Zmiana atrybutów 4
Zmiana stylu CSS 5
4. Obsługa zdarzeń 5
Dodawanie zdarzenia 5
Obiekt zdarzenia 6
Delegowanie zdarzeń 6
5. Tworzenie i usuwanie elementów 7
Tworzenie nowych elementów 7
Usuwanie elementów 7
6. Formularze i manipulacja danymi 8
Odczyt wartości inputów 8
Obsługa wysłania formularza 8

Dodawanie dynamicznych pól 8
7. Skrócona ściąga 9

1. Co to jest DOM?

DOM (Document Object Model) to sposób, w jaki przeglądarka organizuje stronę w formie struktury drzewa, gdzie każdy element HTML jest **węzłem**.

Diagram drzewa DOM



Dlaczego DOM jest ważny

- Pozwala na **dynamiczną** zmianę strony bez jej przeładowania.
- Dzięki DOM możemy **dodawać, zmieniać, usuwać** elementy HTML i reagować na działania użytkownika.

Przykład działania DOM

```
document.body.style.backgroundColor = "lightblue";
```

Powyższy kod ma na celu zmianę koloru tła całego dokumentu na jasno niebieski.

2. Wybieranie elementów HTML

Opis:
Aby manipulować stroną, najpierw musimy **znaleźć** element w DOM.

Najważniejsze metody:

`document.getElementById('id')` – Służy do wybierania elementu HTML po jego
id np. dla elementu:

```
<div id=”text”> Lorem ipsum </div>
```

Kod do pobrania tego elementu poprzez JavaScript wygląda następująco:

```
document.getElementById('text');
```

W nawiasach podajemy nazwę id, zawartą w tagu HTML.

`document.getElementsByClassName('class')` – Zwraca kolekcję (zbiór) wszystkich elementów na stronie, które posiadają wymienioną klasę, np. dla elementu:

```
<div class=”container”> Lorem ipsum </div>
```

Kod do pobrania tych elementów poprzez JavaScript wygląda następująco:

```
document.getElementByClassName(‘container’);
```

W nawiasach podajemy nazwę klasy, zawartą w tagu HTML.

`document.getElementsByTagName('tag')` – Zwraca kolekcję (zbiór) wszystkich elementów na stronie, które posiadają konkretny tag, np. dla elementu:

```
<p> Lorem ipsum </p>
```

Kod do pobrania tych elementów poprzez JavaScript wygląda następująco:

```
document.getElementByTagName(‘p’);
```

W nawiasach podajemy cały tag HTML którym chcemy manipulować.

`document.querySelector('selector')` – Zwraca pierwszy element pasujący do selektora CSS, np. dla elementu:

```
<p class=”text”> Lorem ipsum </p>
```

Kod do pobrania tego elementu poprzez JavaScript wygląda następująco:

```
document.querySelector (‘.text’);
```

W nawiasach podajemy selektor CSS zależny od tego, czy użyliśmy klasy, id czy tagu, dla elementu którym chcemy manipulować.

`document.querySelector('selector')` – Zwraca wszystkie elementy zawierające dany selektor CSS, np. dla elementu:

```
<p class=”text”> Lorem ipsum </p>
```

Kod do pobrania tego elementów poprzez JavaScript wygląda następująco:

```
document.querySelectorAll('p.item')
```

W nawiasach podajemy selektor CSS zależny od tego, czy użyliśmy klasy, id czy tagu, dla elementu którym chcemy manipulować.

3. Modyfikowanie treści i stylów

Opis

Po wybraniu elementu z DOM możemy **modyfikować jego zawartość tekstową**, **atrybuty** oraz **style CSS**. Pozwala to na dynamiczne zmienianie wyglądu i działania strony.

Zmiana tekstu

Możemy zmienić treść widoczną w elemencie.

Przykład:

HTML:

```
<h1 id="tytul"> Stary tytuł </h1>
```

JavaScript:

```
document.getElementById('tytul').textContent = 'Nowy tytuł';
```

Wyjaśnienie:

- **`textContent`** zmienia cały tekst elementu.

Zmiana atrybutów

Możemy dodać lub zmienić dowolny atrybut HTML, np. `href` w linku.

Przykład:

HTML:

```
<a id="mojLink" href="#"> Kliknij tutaj </a>
```

JavaScript:

```
document.getElementById('mojLink').setAttribute('href', 'https://www.google.com');
```

Wyjaśnienie:

- **`setAttribute(nazwa, wartość)`** ustawia atrybut HTML.

Zmiana stylu CSS

Możemy zmieniać wygląd elementów bezpośrednio poprzez JavaScript.

Przykład:

HTML:

```
<p id="tekst"> To jest tekst </p>
```

JavaScript:

```
document.getElementById('tekst').style.color = 'red';
```

```
document.getElementById('tekst').style.fontSize = '20px';
```

Wyjaśnienie:

- **`style`** pozwala modyfikować dowolną właściwość CSS.

4. Obsługa zdarzeń

Opis

Zdarzenia umożliwiają reagowanie na akcje użytkownika takie jak kliknięcia, najechanie myszką, wpisywanie tekstu.

Dodawanie zdarzenia

Przykład:

HTML:

```
<button id="przycisk"> Kliknij mnie </button>
```

JavaScript:

```
document.getElementById('przycisk').addEventListener('click',
function() {
```

```
  alert('Kliknąłeś przycisk!');
});
```

Wyjaśnienie:

- **`addEventListener`** nasłuchuje zdarzenia i wykonuje przypisaną funkcję.

Obiekt zdarzenia

Możemy pobrać dodatkowe informacje o zdarzeniu, np. element, który został kliknięty.

Przykład:

```
document.getElementById('przycisk').addEventListener('click', function(event) {
  console.log(event.target);
});
```

Wyjaśnienie:

- **`event.target`** wskazuje, który element wywołał zdarzenie.

Delegowanie zdarzeń

Technika używana, gdy obsługujemy zdarzenia na wielu dynamicznie tworzonych elementach.

Przykład:

HTML:

```
<ul id="lista">
  <li>Element 1</li>
  <li>Element 2</li>
</ul>
```

JavaScript:

```
document.getElementById('lista').addEventListener('click', function(event) {
  if (event.target.tagName === 'LI') {
    event.target.style.color = 'blue';
```

```
  }
});
```

Wyjaśnienie:

- Delegowanie umożliwia obsługę kliknięcia na wielu elementach bez potrzeby dodawania osobnego nasłuchiwacza na każdy element.

5. Tworzenie i usuwanie elementów

Tworzenie nowych elementów

Przykład:

```
const nowyParagraf = document.createElement('p');
nowyParagraf.textContent = 'To jest nowy paragraf!';
document.body.appendChild(nowyParagraf);
```

Wyjaśnienie:

- **`createElement('tag')`** tworzy nowy element HTML.
- **`appendChild(element)`** dodaje element do dokumentu.

Usuwanie elementów

Przykład:

HTML:

```
<div id="doUsuniecia"> Usuń mnie </div>
```

JavaScript:

```
document.getElementById('doUsuniecia').remove();
```

Wyjaśnienie:

- **`remove()`** natychmiast usuwa element z DOM.

6. Formularze i manipulacja danymi

Odczyt wartości inputów

Przykład:

HTML:

```
<input type="text" id="imie" placeholder="Wpisz imię">
```

JavaScript:

```
const imie = document.getElementById('imie').value;
console.log(imie);
```

Wyjaśnienie:

- **`value`** zwraca aktualną wartość wpisaną w polu **`input.`**

Obsługa wysłania formularza

Przykład:

HTML:

```
<form id="formularz">
  <input type="text" id="email" placeholder="Email">
  <button type="submit">Wyślij</button>
</form>
```

JavaScript:

```
document.getElementById('formularz').addEventListener('submit',
function(event) {
  event.preventDefault();
  alert('Formularz został wysłany!');
});
```

Wyjaśnienie:

- **`preventDefault()`** *zatrzymuje domyślne działanie (np. przeładowanie strony po wysłaniu formularza)***.**

Dodawanie dynamicznych pól

Przykład:

```
const formularz = document.getElementById('formularz');
const nowePole = document.createElement('input');
nowePole.type = 'text';
nowePole.placeholder = 'Nowe pole';
formularz.appendChild(nowePole);
```

Wyjaśnienie:

- *Dynamiczne dodawanie inputów do istniejącego formularza*.

7. Skrócona ściąga

Akcja	**Metoda**
Znalezienie elementu po ID	getElementById('id')
Znalezienie elementu po klasie	getElementsByClassName('class')
Znalezienie po selektorze	querySelector('selector')
Zmiana tekstu	textContent
Zmiana atrybutu	setAttribute('attr', 'value')
Zmiana stylu	style.property = value
Dodanie zdarzenia	addEventListener('event', function)
Utworzenie nowego elementu	createElement('tag')
Usunięcie elementu	remove()